Zookeeper

了解如何用 ZooKeeper 解決系統設計中的大量協調問題。

協調分散式系統很難。運算能力與擴展方式已經大幅演進，但根本問題還在：要怎麼讓幾十、幾百台伺服器順暢地一起工作？當這些機器需要選舉 leader、維護一致設定、即時偵測故障時，你面對的正是 ZooKeeper 被設計來解決的問題。

ZooKeeper 在 2008 年釋出，已經有些年頭，也出現不少替代方案。但在 Apache 生態系裡，它仍然扮演核心角色。

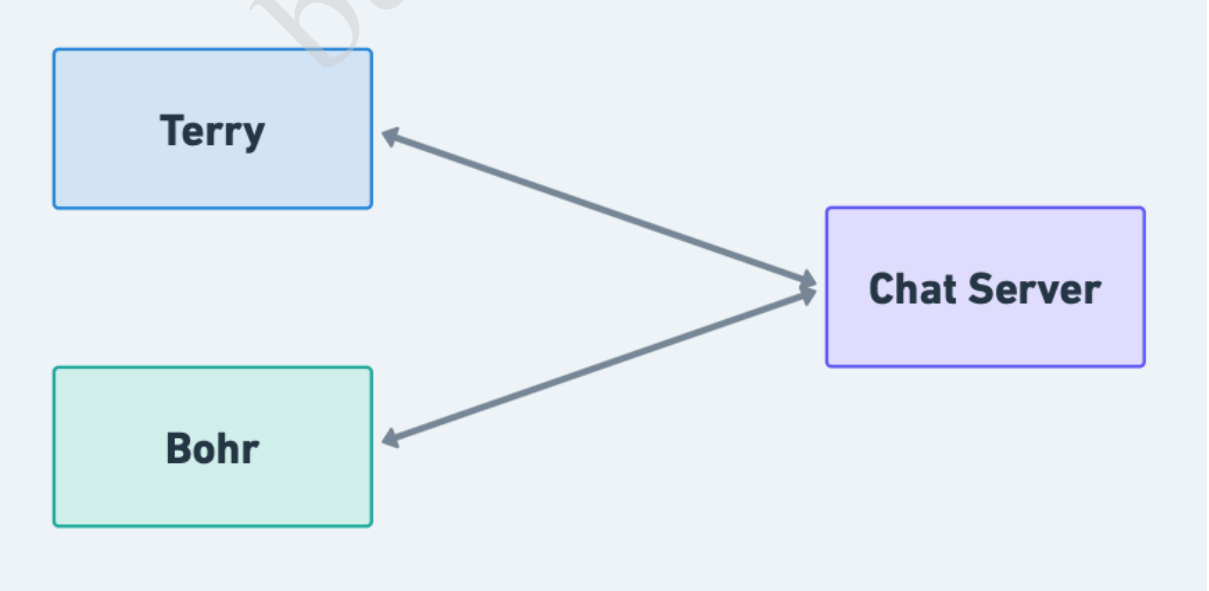
儘管如此，理解 ZooKeeper 能讓你掌握分散式系統裡那些通用概念，就算你之後不會直接用到它。透過了解 ZooKeeper 如何用簡單的 primitives（階層式命名空間、資料節點、watches）做協調，你會對共識、leader 選舉、設定管理等通用問題有更清楚的認識。

下面我們會走過 ZooKeeper 怎麼運作、什麼時候該用、以及它在當代分散式系統中的演變。

一個動機範例

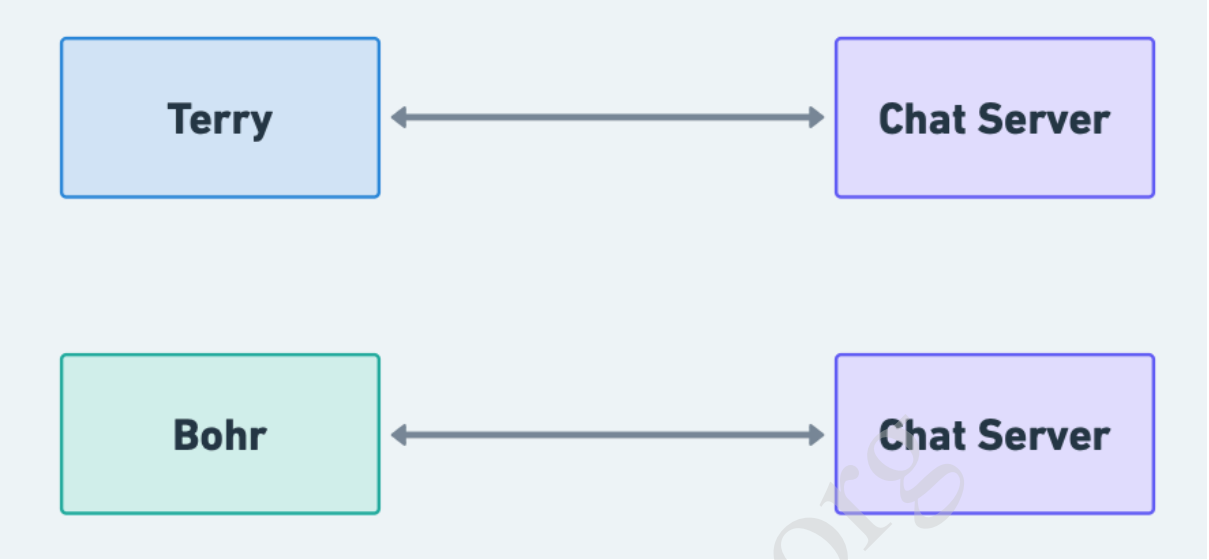
要理解為什麼協調很難，先從一個例子開始。假設你在做一個聊天應用。

一開始聊天應用只跑在一台伺服器上。一切很單純。當 Terry 傳訊息給 Bohr，兩人都連到同一台伺服器。伺服器知道要把訊息送到哪裡，全部在記憶體裡、延遲低、不需要協調。



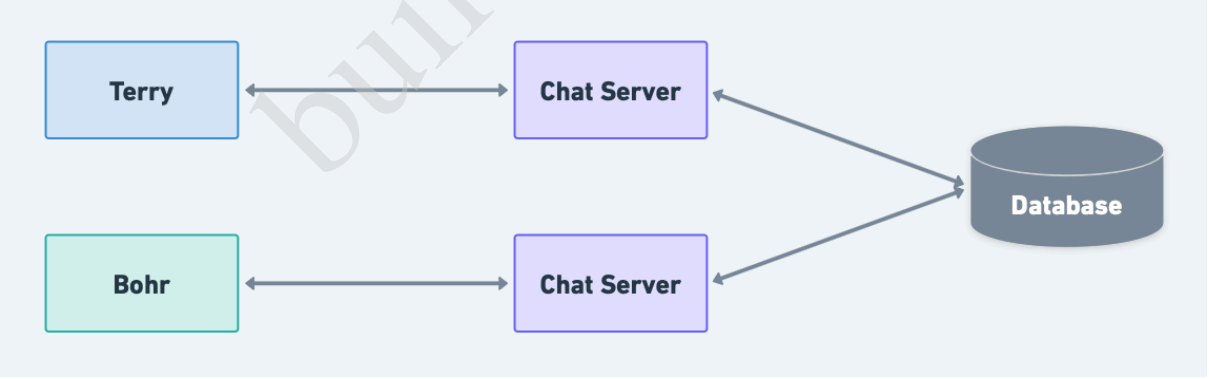
buildnoat.org

但應用變熱門了。單一伺服器撐不住，你加第二台。問題來了：當 Terry（連到 Server 1）要傳訊息給 Bohr（連到 Server 2），Server 1 必須知道 Bohr 在哪裡才能轉發。Server 1 要怎麼發現 Bohr 連到哪台伺服器？



你可能會想：「用資料庫就好了！」建一張表，把使用者對應到伺服器。使用者連線時更新資料庫，要轉發訊息時查資料庫。聽起來很簡單。

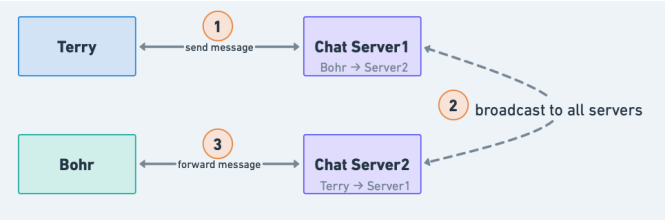
但這會帶來新問題。資料庫變成單點故障，一掛掉整個聊天系統就壞了。而且每次轉發都要先查一次資料庫，延遲變高。當系統擴到幾百萬使用者，這顆資料庫會被查詢打爆，變成瓶頸。



你也許會用快取優化，但接著要處理快取一致性。要是使用者從 Server 2 斷線後連到 Server 3，而 Server 1 的快取還以為他在 Server 2 呢？訊息就會送錯或丟失。

於是你想：「讓伺服器彼此直接通知變化！」使用者連到 Server 3 時，Server 3 對所有其他伺服器廣播。但當系統長到幾十、幾百台伺服器，這種廣播會產生大量網路流量。每台伺服器都要和每台其他伺服器保持連線，也就是 n^2 條連線，擴展性很差。

buildroot.org



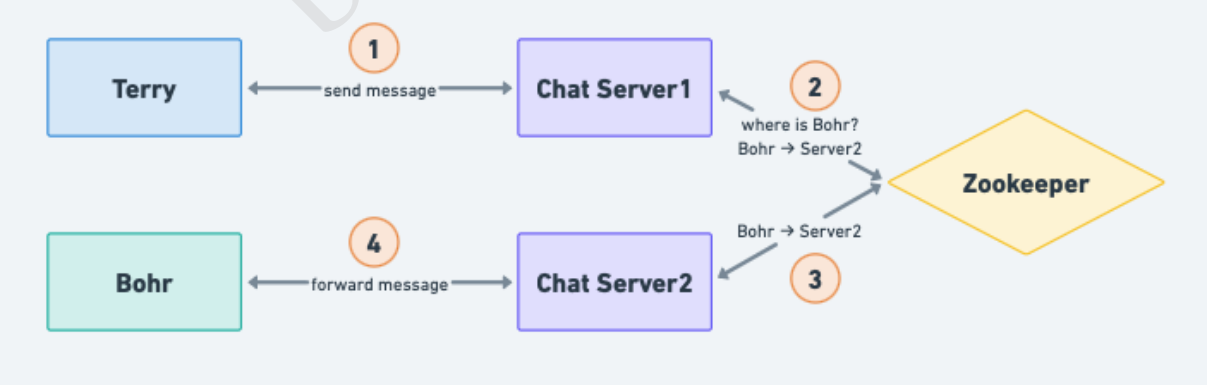
然後還有伺服器故障問題。要是 Server 2 當機了呢？它沒辦法告訴任何人它掛了。連到它的使用者會斷線，但其他伺服器不知道，還會繼續把訊息往已經死掉的伺服器送。

於是你做 heartbeats：伺服器週期性地檢查其他伺服器是否還活著。但這又引出新問題：有名的「網路分區」問題。要是 Server 2 其實還活著，只是和 Server 1 之間網路有問題呢？Server 1 會以為 Server 2 死了，其實沒有。有些伺服器可能認為 Server 2 活著，有些認為死了，你對系統狀態的認知就不一致了。

聊天應用當然也可以用 pub/sub 模式避開部分協調問題，但這裡我們刻意聚焦在「必須直接協調」的情境，以便說明 ZooKeeper 能做的事。很多真實系統確實需要我們在討論的這類協調。

這些問題——服務發現、設定共享、故障偵測、leader 選舉、分散式共識——都是分散式系統的基本挑戰。要正確解決它們非常難，需要能處理部分故障、網路延遲、同時維持一致性的演算法。而 ZooKeeper 正是為解決這組問題而生的。

ZooKeeper 能解決上面討論的所有問題。它提供一個一致、可靠的「真相來源」，所有伺服器都可以信任。伺服器上線時在 ZooKeeper 註冊；使用者連到某台伺服器時，這個對應關係存在 ZooKeeper。ZooKeeper 會通知有興趣的伺服器變化，並透過 ephemeral nodes 自動處理故障。你不需要自己實作這些複雜的分散式演算法，就能得到可靠的服務發現、設定管理與 leader 選舉。



接著我們具體看 ZooKeeper 怎麼處理這些挑戰。

ZooKeeper 基礎

核心上，ZooKeeper 提供一組簡單但強大的 primitives，用來解決複雜的分散式協調問題。要理解它怎麼運作，需要掌握三個概念：以 ZNode 為基礎的資料模型、ensemble 裡的伺服器角色、以及用於即時通知的 watch 機制。

可以把 ZooKeeper 想成一個「同步的 metadata 檔案系統」——每個連上的節點看到的都是同一份資料。正是這種「所有參與伺服器都有一致視圖」的特性，讓 ZooKeeper 在協調任務上如此有用。

資料模型：ZNode

ZooKeeper 的資料組織成階層式命名空間，很像檔案系統或一棵樹。樹上的節點叫 ZNode。和傳統檔案系統的資料夾不同，ZNode 可以存資料（通常很小，小於 1MB），並帶有 metadata。

每個 ZNode 由一個 path 識別，類似檔案路徑。例如 /app1/config 可以代表名叫「app1」的應用程式設定。重點是：ZNode 是拿來存「協調用資料」的，不是大量資料（例如圖片或大文件），所以單一 ZNode 的資料通常很小，而 ZNode 的數量通常很大（成千上萬）。

ZNode 主要有三種，在我們的聊天應用裡各有用途：

Persistent ZNode：這種節點會一直存在，直到被明確刪除。在聊天應用裡，我們用 persistent nodes 存設定，例如最大訊息大小、限流參數等。

```
# Store the maximum message size in bytes
create /chat-app/config/max_message_size "1024"
```

Ephemeral ZNode：建立它的 session 結束時（不論是客戶端斷線或逾時），節點會自動被刪除。很適合用來表示「哪台伺服器還活著、哪個使用者還在線」。

```
# Server 2 registers itself when it starts up
create -e /chat-app/servers/server2 "192.168.1.102:8080"

# When Bohr connects to Server 2, it creates:
create -e /chat-app/users/bohr "server2"

# If Server 2 crashes, both nodes automatically disappear!
```

Sequential ZNode：名稱會自動加上單調遞增的序號。在聊天應用裡可以用來為訊息排序或實作分散式鎖。

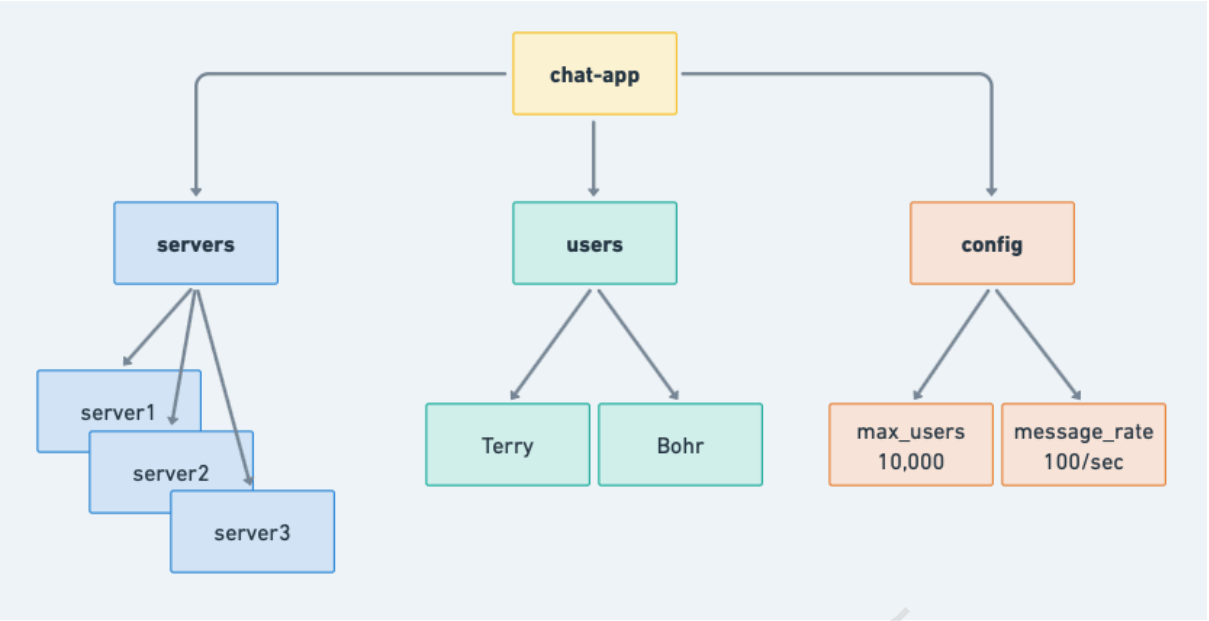
buildmoat.org

```
# Terry sends a message to the global chat
create -s /chat-app/channels/global/msg- "Terry: Hello ever
yone!"
# Creates /chat-app/channels/global/msg-0000000001
```

在我們的聊天應用裡，可以想像這樣的結構：

```
/chat-app
  /servers           # Directory of available servers
    /server1         # Ephemeral node containing "192.168.
1.101:8080" the location of this server
    /server2         # Ephemeral node containing "192.168.
1.102:8080" the location of this server
    /server3         # Ephemeral node containing "192.168.
1.103:8080" the location of this server
  /users             # Directory of online users
    /terry           # Ephemeral node containing "server1"
the server that terry is connected to
    /bohr             # Ephemeral node containing "server2"
the server that bohr is connected to
  /config            # Application configuration
    /max_users       # Persistent node containing "10000" t
he maximum number of users allowed
    /message_rate    # Persistent node containing "100/sec"
the maximum number of messages per second allowed
```

buildnoat.org

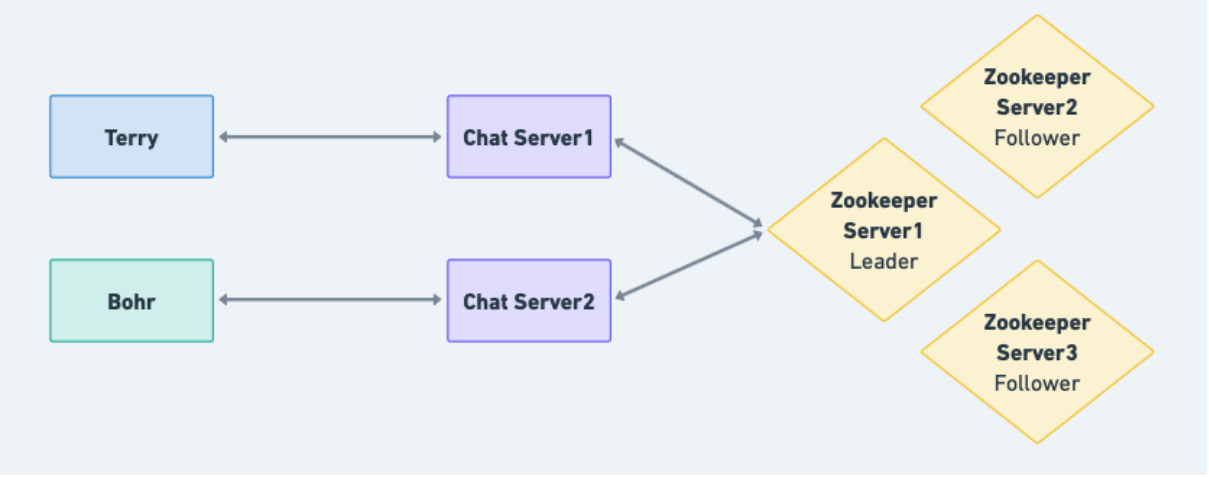


這樣就優雅地解決了協調問題。當 Terry（在 Server 1）要傳訊息給 Bohr，Server 1 只要查 /chat-app/users/bohr 就知道 Bohr 在 Server 2，然後把訊息轉過去。如果 Bohr 斷線後連到 Server 3，ZNode 會自動更新，訊息永遠會轉到正確的伺服器。

實務上，熱門聊天應用可能同時有幾百萬人在線，我們不會想為每個使用者建一個 ZNode。可以改用 consistent hashing：伺服器在 ZooKeeper 註冊，使用者則用 user ID 的 hash 對應到伺服器。這樣 ZooKeeper 只需要追蹤伺服器，不用追蹤幾百萬個使用者，擴展性更好，同時仍能透過 ephemeral server nodes 快速偵測故障。

伺服器角色與 Ensemble

ZooKeeper 不是設計成只跑在一台機器上的，那樣會變成單點故障。它跑在一群伺服器上，這群伺服器叫 ensemble。常見的生產環境會用 3、5 或 7 台（奇數在「多數決」時比較好處理）。



buildnear.org

在 ensemble 裡，伺服器扮演不同角色：

Leader：會選出一台當 leader，負責處理所有寫入請求。當 Server 2 要在 ZooKeeper 註冊新的使用者連線時，這個寫入請求會送到 leader。

Followers：其餘伺服器跟隨 leader 的指示，並協助處理讀取請求。當 Server 1 要查 Bohr 連到哪台時，可以向 ensemble 裡任一台 ZooKeeper 伺服器讀取。

這種分散式設計解決了「用資料庫做使用者–伺服器對應」時的單點故障問題。只要多數（quorum）的伺服器還在，即使有一台 ZooKeeper 掛掉，ensemble 仍能運作。例如 3 台組成的 ensemble 可以容忍 1 台故障。

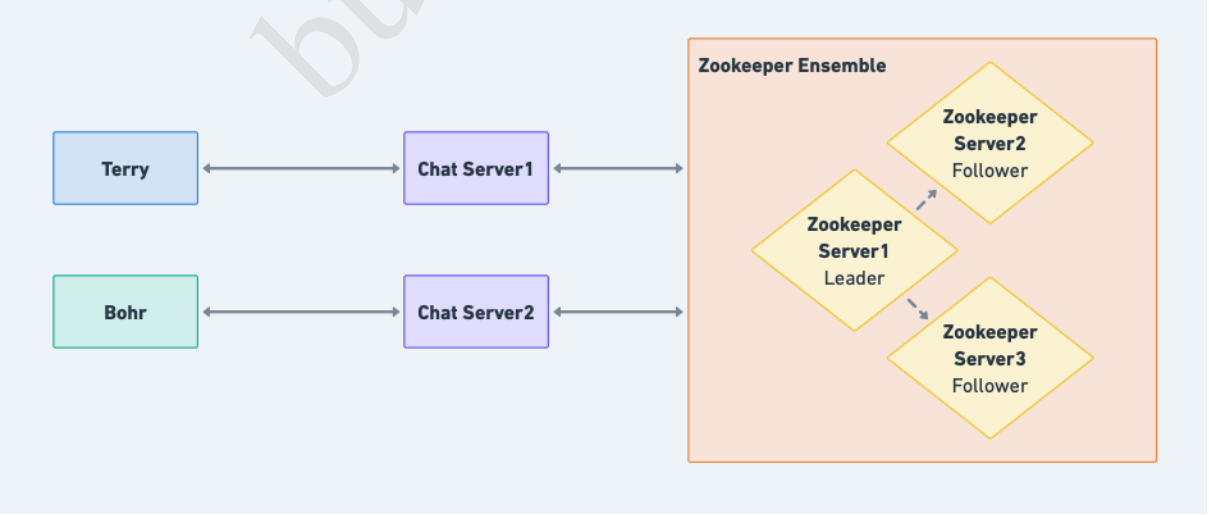
聊天伺服器連到 ZooKeeper 時，會連到 ensemble 裡的所有伺服器：

```
// Chat Server 1 connecting to ZooKeeper
ZooKeeper zk = new ZooKeeper("zk1:2181,zk2:2181,zk3:2181",
                             3000, /* session timeout */
                             watcher /* callback */);
```

這種 ensemble 設計讓關鍵的協調資料——誰連到哪台伺服器——在個別 ZooKeeper 節點故障時仍能保持高可用與持久。

Watches：知道什麼時候變了

ZooKeeper 最強大的功能之一是 watch 機制，它優雅地解決了聊天應用裡的「通知」問題。Watches 讓伺服器在 ZNode 變動時收到通知，不需要不斷輪詢或複雜的伺服器間通訊。



沒有 watcher 的話，每次有新訊息要送，伺服器都得去問 ZooKeeper 收件人在哪，延遲會變大，也會對 ZooKeeper 叢集造成很大壓力。在每秒幾千則訊息的規模下，

buildmoat.org

ZooKeeper 很快就會從「解方」變成瓶頸。

在聊天應用裡，watches 這樣幫忙：

Server 1 啟動時，對 /chat-app/users 目錄設一個 watch：

```
// Server 1 watching for user changes
zk.getChildren("/chat-app/users", true, null);
```

當 Bohr 從 Server 2 斷線並連到 Server 3，Server 3 會更新 Bohr 的 ZNode：

```
// Server 3 updates Bohr's location
zk.setData("/chat-app/users/bohr", "server3".getBytes(), -
1);
```

ZooKeeper 會透過 watcher 回呼自動通知 Server 1：

```
// Server 1's watcher callback
public void process(WatchedEvent event) {
    if (event.getType() == EventType.NodeDataChanged &&
        event.getPath().equals("/chat-app/users/bohr")) {
        // Get updated server for Bohr
        byte[] data = zk.getData("/chat-app/users/bohr", tr
ue, null);
        String bohrsServer = new String(data);
        // Update routing table: Bohr is now on Server 3
        routingTable.put("bohr", bohrsServer);
    }
}
```

這個 watch 機制取代了我們之前考慮的複雜廣播。不用每台伺服器都和每台其他伺服器保持連線（n^2 條），大家只要連到 ZooKeeper。當使用者從一台伺服器換到另一台，只有關心該使用者的伺服器會收到通知。

那伺服器故障呢？如果 Server 2 當機，它和 ZooKeeper 的 session 會結束，它建立的所有 ephemeral nodes（包括連線使用者的那些）都會自動被刪除。正在 watch 這些節點的其他伺服器會收到通知，可以做對應處理，例如把那些使用者標成離線。

重要的是，watcher 支援一種常見模式：伺服器在本地快取 ZooKeeper 的狀態。當伺服器想知道 Bohr 在哪，不必每次都問 ZooKeeper，查本地快取就好。若有變動，伺服器會被通知並更新快取。

buildneat.org

這也是為什麼 ZooKeeper 被稱為「協調服務」而不是「資料庫」——它設計來「通知系統有變化」，不是處理高流量的讀取。

結合這三個基本概念——用 ZNode 存資料、可靠的伺服器 ensemble、以及用 watches 做變更通知——ZooKeeper 形成一個強大的基礎，優雅地解決我們在聊天應用裡遇到的所有分散式協調難題。

主要能力

我們用聊天應用當例子，但 ZooKeeper 的用途不限於此。它主要能支援四類能力：設定管理（Configuration Management）、服務發現（Service Discovery）、Leader 選舉（Leader Election）、以及分散式鎖（Distributed Locks）。下面分別看它怎麼運作。

用 ZooKeeper 做 Configuration Management

ZooKeeper 最常見的用途之一，是在分散式系統裡儲存與散佈設定。例如資料庫連線字串、feature flags、服務端點等。

在聊天應用裡我們已經看過可以這樣存設定：

```
/chat-app/config/max_message_size "1024"
/chat-app/config/message_rate "100/sec"
```

真正的威力在於：當設定變更時，ZooKeeper 能通知所有有興趣的服務。假設你想在整個聊天平台上開啟一個新功能。用 ZooKeeper 你可以：

更新單一 ZNode：set /chat-app/config/enable_reactions “true”

所有正在 watch 這個節點的聊天伺服器都會收到通知

伺服器不用重啟就能更新行為

這形成強大的集中式設定管理，因為能即時傳播、版本化、以及原子更新。

電商平台可以這樣用 ZooKeeper 做設定：

```
/ecommerce
  /config
    /pricing_algorithm "dynamic_v2"  # Switch pricing algor
ithm across all services
    /discount_threshold "50.00"      # Update discount thre
shold in real-time
```

buildmoat.org

```
    /maintenance_mode "false"        # Toggle maintenance m
ode off/on
```

用 ZooKeeper 做設定時，重點放在「執行時可能會變的動態設定」。只在部署時才變的靜態設定，往往放在檔案或環境變數就好。

多數現代雲端廠商都有自家的設定管理方案，例如 AWS 有 AWS Systems Manager Parameter Store，Azure 有 Azure App Configuration。若在單一雲端廠商內開發，可以優先考慮用他們的原生方案，而不是自己架 ZooKeeper。

用 ZooKeeper 做 Service Discovery

Service discovery 是在分散式系統裡自動發現服務與端點的過程。服務上線時註冊自己，下線時註銷（或它的 ephemeral nodes 過期）。

我們在聊天伺服器註冊自己的例子裡已經看過：

```
create -e /chat-app/servers/server2 "192.168.1.102:8080"
```

這種模式支援動態擴縮、負載平衡與健康檢查。

以線上串流平台的微服務架構為例：

```
/streaming
  /services
    /video-transcoder
      /instance1 "10.0.0.1:8080"
      /instance2 "10.0.0.2:8080"
    /recommendation-engine
      /instance1 "10.0.1.1:9000"
      /instance2 "10.0.1.2:9000"
    /payment-processor
      /instance1 "10.0.2.1:5000"
```

當新的影片上傳服務需要找可用的 transcoder，只要：

讀取 /streaming/services/video-transcoder 的子節點

連到其中一個可用的 transcoder 實例

設一個 watch，以便在可用 transcoder 有變動時收到通知

現在很多系統會用專門的 service discovery 工具，例如 Consul 或 etcd，或依賴平台提供的方案如 Kubernetes Services、AWS Service Discovery。但這些工具實作的模

buildmoat.org

式與 ZooKeeper 非常類似，有些甚至在內部仍使用 ZooKeeper。

用 ZooKeeper 做 Leader Election

在分散式系統裡，常常需要指定一個節點當「leader」，負責某些操作。例如可能只想讓一台伺服器處理付款交易或排程工作。有 leader 可以協調這些操作，確保同一時間只有一台在做。

ZooKeeper 的 sequential ZNode 讓 leader 選舉變得很直接：

每台伺服器在指定路徑下建立一個 sequential ephemeral node

序號最小的那台成為 leader

其餘伺服器 watch 序號次小的那個節點

若 leader 掛了，它的節點消失，序號次小的那台就會接替

在聊天應用裡，如果我們要選出一台伺服器負責全域公告，可以這樣做：

```
# Server 1 creates:
create -s -e /chat-app/leader/node- "server1"  # Creates /c
hat-app/leader/node-0000000001

# Server 2 creates:
create -s -e /chat-app/leader/node- "server2"  # Creates /c
hat-app/leader/node-0000000002

# Server 3 creates:
create -s -e /chat-app/leader/node- "server3"  # Creates /c
hat-app/leader/node-0000000003
```

Server 1 序號最小，所以是 leader。Server 2 watch node-0000000001，Server 3 watch node-0000000002。若 Server 1 掛掉，它的節點消失，Server 2 會收到通知並成為新 leader，Server 3 改為 watch Server 2。

這個模式提供自動 failover，並保證同一時間只有一台在執行關鍵操作。

同樣的做法也用在 HBase（需要一台協調 schema 變更）、以及早期 Kafka（由 controller broker 管理 partition leadership）等系統。

用 ZooKeeper 做 Distributed Locks

分散式鎖讓不同機器上的多個行程能協調對共享資源的存取，在分散式系統裡用來避免競態條件。

buildmoat.org

ZooKeeper 用 sequential ephemeral ZNode 實作分散式鎖：

每個想拿鎖的客戶端在鎖的路徑下建立一個 sequential ephemeral node

所有客戶端依序號排序這些節點

序號最小的客戶端持有鎖

每個客戶端 watch 序號比它小一號的那個節點

當客戶端釋放鎖（或當機）時，它的 ZNode 被移除，下一個客戶端會收到通知

在聊天應用裡，可以想像對「發送訊息」做限流：

```
# Client 1 wants to send a message:
create -s -e /chat-app/locks/send_message- "client1"  # Cre
ates /chat-app/locks/send_message-0000000001

# Client 2 also wants to send:
create -s -e /chat-app/locks/send_message- "client2"  # Cre
ates /chat-app/locks/send_message-0000000002

# Client 1 sends its message, then deletes its node
delete /chat-app/locks/send_message-0000000001

# Client 2 is notified and now holds the lock
```

這種分散式鎖模式可用在資源分配、並行控制、叢集排程等情境。

ZooKeeper 的鎖很好用，但不是為高頻上鎖（每秒幾百次）設計的。那種情境可以考慮專門方案，例如基於 Redis 的鎖或資料庫交易。

什麼時候選 ZooKeeper 鎖、什麼時候選 Redis 鎖？

Redis 分散式鎖效能和實作簡單度較佳。當你需要更強的一致性保證、且正確性比效能重要時（例如金融交易），應該選 ZooKeeper 鎖。ZooKeeper 也比較適合長時間持有的鎖（例如數小時），因為透過 ephemeral nodes 的自動故障偵測，在伺服器當機時的處理比 Redis 鎖更穩健；Redis 鎖需要仔細處理逾時與 heartbeat。

ZooKeeper 怎麼運作

我們已經知道 ZooKeeper 能做什麼，接下來看它底層是怎麼做到的。

老實說，面試裡需要深到這層的機率不高。但這裡的很多概念在分散式系統裡非常通用，即使脫離 ZooKeeper 也值得理解。若你覺得負擔太大，可以先跳過這節。

buildmoat.org

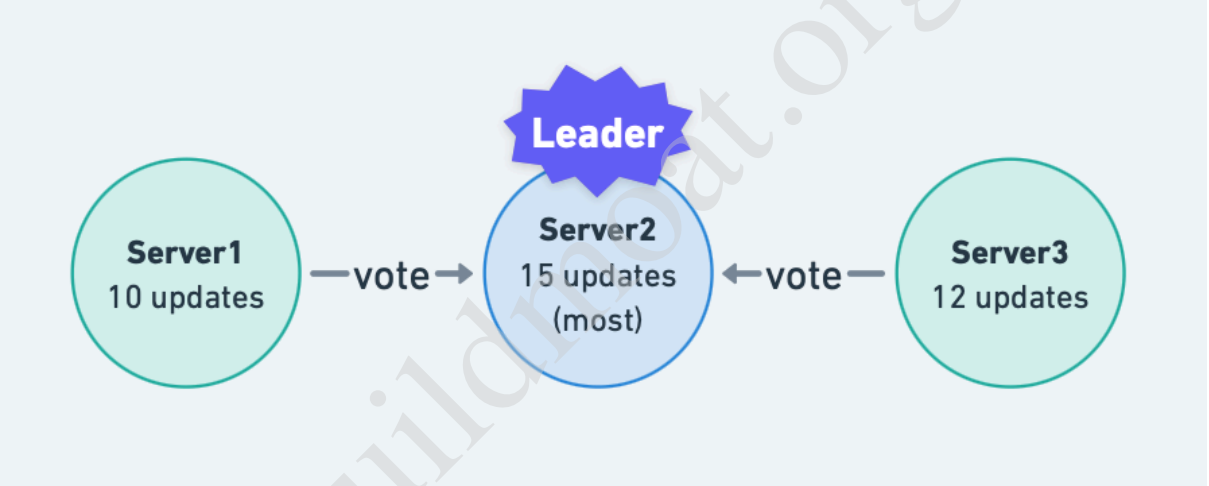
用 ZAB 達成共識

前面說過，ZooKeeper 本身是由多台伺服器組成的 ensemble，也就是多台機器跑同一套 ZooKeeper、彼此協調。有個有趣的對比：ZooKeeper 幫其他應用解決協調問題，但它自己作為分散式系統，也得先解決自己的協調問題。

ZooKeeper 的核心是 ZooKeeper Atomic Broadcast（ZAB）協定。ZAB 讓所有 ZooKeeper 伺服器在即使有節點故障或網路問題時，仍能對系統狀態達成一致。

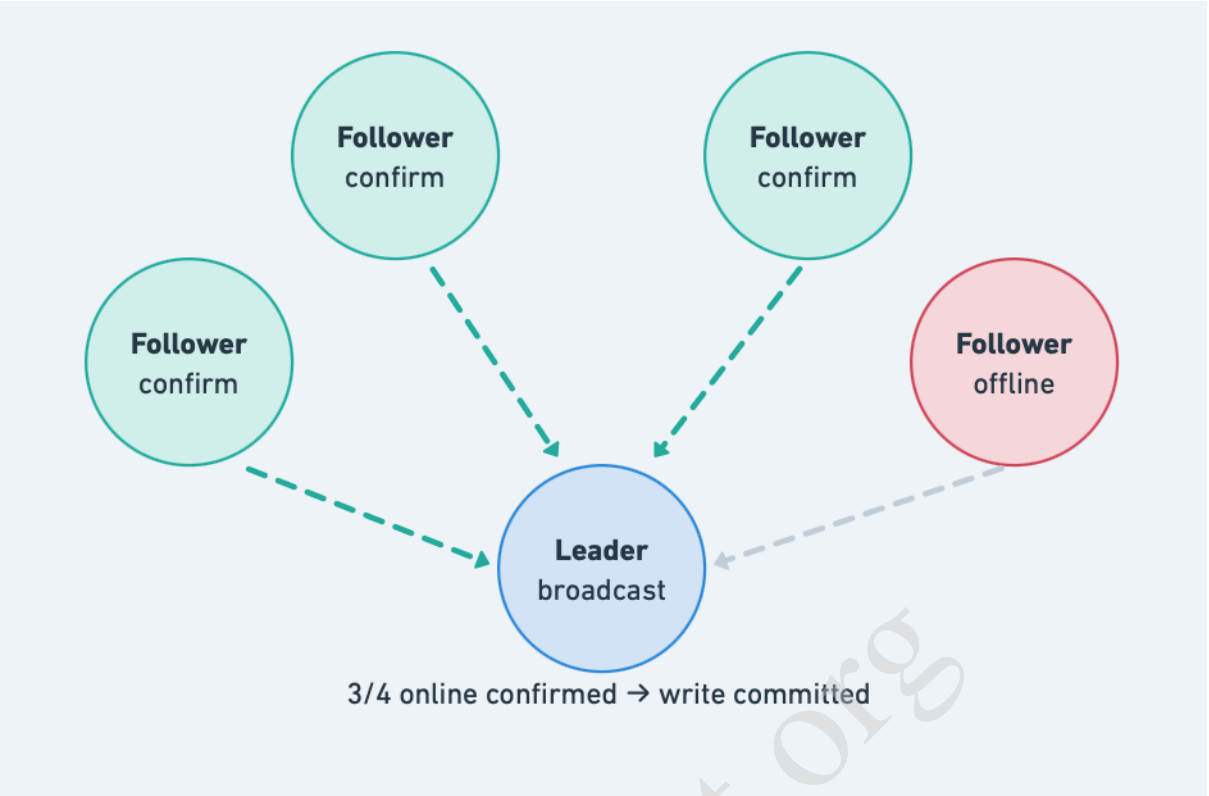
ZAB 有兩個主要階段：

Leader Election：當 ZooKeeper ensemble 啟動，或現任 leader 掛掉時，伺服器會用投票選出新 leader。選舉時主要看誰擁有最新的 transaction 歷史。若多台伺服器的 transaction 歷史一樣新，則 ID 最高的那台會被選上。（注意：這和應用層的 leader 選舉相反——應用層是序號最小的 sequential ZNode 當 leader，正好說明 ZooKeeper 內部機制和建在它上面的模式可以不同。）



Atomic Broadcast：選出 leader 後，所有寫入請求都送到 leader。Leader 再把變更 broadcast 給所有 followers。只有當多數（quorum）伺服器都持久化這個變更後，寫入才算成功。

buildmoat.org



在聊天應用裡，當 Server 2 要為新使用者 Bohr 建立一個 ZNode 時，流程是：

1. Server 2 → ZooKeeper Leader：「在 /chat-app/users/bohr 建立一個 ZNode，值為 'server2'」
2. ZooKeeper Leader → 所有 Followers：「我們來加這個 Bohr 的 ZNode」
3. Followers → Leader：「變更已接受並寫入」（需要多數）
4. Leader → Server 2：「Bohr 的 ZNode 已成功建立」

ZAB 和你可能聽過的 Paxos、Raft 等共識演算法類似。ZAB 比 Raft 早，實作細節有差異，但高層目標一樣：在分散式系統裡達成共識。

這個兩階段協定確保即使部分伺服器當機或發生網路分區，剩下的伺服器仍能維持一致的資料視圖。Quorum（多數）要求代表：5 台組成的 ensemble 可以容忍 2 台故障仍正常運作。

強一致性保證

ZooKeeper 提供幾項重要的一致性保證，讓它適合作為分散式協調的可靠基礎：

Sequential Consistency：同一個客戶端送出的更新，會依送出順序被套用。若客戶端先更新 node A 再更新 node B，所有伺服器都會先看到 A 的更新再看到 B。

Atomicity：更新要嘛全部成功，要嘛全部失敗，沒有「只寫一半」的狀態。

buildmoat.org

Single System Image：不論客戶端連到哪台伺服器，在同步完成後看到的都是同一份系統視圖。客戶端可以向任一台讀取，但所有寫入都經過 leader。

Durability：一旦更新被套用，就會一直存在直到被客戶端覆寫。即使伺服器故障重啟，更新也不會丟失。

Timeliness：系統保證客戶端對系統的視圖會在有限時間內更新。

ZooKeeper 透過 ZAB 達成這些保證。所有寫入都經過 leader，形成全序的更新，且只有在 quorum 的伺服器把變更寫入 transaction log 後才視為成功。協定使用兩階段提交：leader 提出更新、等待確認、然後才 commit，這樣在伺服器故障時仍能保證原子性與持久性。

為了維持 single system image，ZooKeeper 為每個 ZNode 維護版本號，並用同步協定讓 followers 在斷線重連後追上。但從 follower 讀取可能拿到較舊的資料，因為讀取不會每次都問 leader。若需要最強一致性，客戶端可以在讀取前先做「sync」操作，確保拿到最新資料。

ZooKeeper 提供強一致性，但不是為高吞吐讀寫設計的。它比較適合「讀多寫少」、且資料量相對小的負載。

讀寫操作

ZooKeeper 的架構特別針對「讀多」的負載做了優化，這會影響讀寫的處理方式：

讀取：ensemble 裡任一台伺服器都可以直接用自己記憶體裡的資料副本服務讀取請求。所以讀取吞吐高、延遲低，ZooKeeper 在「讀寫比大約 10:1」的負載下表現很好。

寫入：所有寫入請求都必須送到 leader，由 leader 透過 ZAB 在 ensemble 內協調更新。這種集中式做法保證了順序一致，但也讓寫入比讀取貴很多。

在聊天應用裡就是：

當 Server 1 要查 Bohr 連到哪（讀取），可以向任一台 ZooKeeper 查。

當 Server 3 要更新 Bohr 的位置（寫入），請求會到 leader，再由 leader 同步到整個 ensemble。

因為讀取是各台本地服務的，若客戶端連到一個還沒跟 leader 同步完的 follower，可能讀到舊資料。對需要最強一致性的應用，ZooKeeper 提供「sync」操作，在讀取前先確保該伺服器已跟上。

Session 與連線管理

ZooKeeper 用 session 的概念管理客戶端連線與 ephemeral nodes。Session 對偵測客戶端故障、以及實作 ephemeral nodes 等功能都很關鍵。

buildmoat.org

Session 建立：客戶端連上 ZooKeeper 時會建立一個 session，逾時時間可設定（常見 10–30 秒）。

Heartbeats：客戶端週期性送 heartbeat 維持 session。若 ZooKeeper 在逾時時間內沒收到 heartbeat，就視為客戶端故障。

Session 恢復：若客戶端和某台 ZooKeeper 的連線斷了，只要在 session 逾時前連到另一台，可以恢復同一個 session。

Session 過期：若 session 過期，該客戶端建立的所有 ephemeral nodes 會自動被刪除，該客戶端註冊的所有 watches 也會被移除。

在聊天應用裡，這個 session 機制在伺服器或用戶意外斷線時會自動清理：

1. Server 2 啟動時和 ZooKeeper 建立 session
2. 它為自己與連上的使用者建立 ephemeral nodes
3. 若 Server 2 當機，它的 session 最終會逾時
4. ZooKeeper 自動刪除 Server 2 擁有的所有 ephemeral nodes
5. 正在 watch 這些節點的其他伺服器會收到使用者離線的通知

把 session timeout 設對很重要。太短，暫時的網路問題可能造成不必要的 session 過期；太長，系統要比較久才能偵測到真正的故障。

儲存架構

持久性怎麼保證？ZooKeeper 如何確保資料不丟失，尤其在 ensemble 裡有節點當機時？

Transaction Log：每個狀態變更（transaction）會先寫入持久儲存上的 transaction log。這種 write-ahead logging 確保已確認的更新不會丟失，即使伺服器在確認後馬上當機。

Snapshots：ZooKeeper 會週期性對記憶體裡的資料庫做 snapshot，加速恢復。伺服器重啟時會載入最近的 snapshot，再重放 transaction log 還原完整狀態。

ZooKeeper 文件特別強調 transaction log 是效能關鍵：

「ZooKeeper 最影響效能的部分是 transaction log。ZooKeeper 必須在回覆前把 transaction 同步到儲存媒體。使用獨立的 transaction log 裝置是維持穩定效能的關鍵。」

記憶體被 swap 會嚴重影響 ZooKeeper，因為所有操作都是有序的。只要有一個請求因為 swap 打到磁碟，後面排隊的請求都會被拖慢。適當設定 heap 大小以避免 swap 很重要。

buildmoat.org

故障處理

如果 ensemble 裡有一台 ZooKeeper 伺服器故障會怎樣？

伺服器故障：若是一台 follower 掛了，只要還有 quorum 的伺服器在，leader 可以繼續運作。若是 leader 掛了，會自動觸發 leader 選舉，由 ID 最高的 follower 接任。

網路分區：若網路分區導致沒有任何一組能形成多數，ZooKeeper 不會處理寫入請求，直到分區恢復。這避免了「split-brain」——系統不同部分對當前狀態看法不一致。

客戶端故障：若客戶端故障，它建立的所有 ephemeral nodes 會在它的 session 逾時後自動被刪除。對聊天應用來說，若 Server 2 當機，連到它的所有使用者在對應的 ZNode 消失後會自動被標為離線。

客戶端 Session 管理：ZooKeeper 追蹤客戶端 session，並提供在暫時斷線後重新連線、恢復 session 與 watches 的機制。

在聊天應用裡，當一台伺服器故障時會發生：

1. Server 2 當機
2. Server 2 的 ZooKeeper session 逾時（通常 10–30 秒）
3. Server 2 建立的所有 ephemeral nodes 自動被刪除，包括 /chat-app/servers/server2 以及所有「連到 Server 2 的使用者」的 /chat-app/users/X
4. 正在 watch 這些節點的其他伺服器收到通知
5. 它們更新自己的 routing table，把那些使用者標為離線
6. 當新的 Server 3 上線，會建立新 session 並註冊自己

這種自動故障處理是 ZooKeeper 最強大的特性之一，讓分散式系統能在無人為干預下從故障恢復，並維持一致性。

ZooKeeper 的 session timeout 是關鍵參數。設太低，暫時的網路問題可能造成不必要的 failover；設太高，系統要比較久才能偵測並反應真正的故障。

ZooKeeper 在現代的位置

ZooKeeper 仍是分散式協調的重要一員，但自 2008 年問世以來，生態已經變化很多。了解 ZooKeeper 在當代分散式系統中的定位，有助於在面試裡做出更好的設計選擇，也避免提出可能過時的方案。ZooKeeper 仍是久經考驗的工具，但已不是唯一選項。

在主要分散式系統中的現況

pulumicat.org

ZooKeeper 在 Apache 生態裡仍然很常見，是 HBase、Hadoop、SolrCloud、Storm、NiFi、Pulsar 等專案的核心組件。

其他重要系統如 ClickHouse 在複製協調、分散式 DDL 執行、以及複製環境的 metadata 儲存上也需要 ZooKeeper。

Kafka 近年從 ZooKeeper 遷移出去，是分散式系統演進的一個重要轉折。多年依賴 ZooKeeper 做協調後，Kafka 引入了 Kafka Raft Metadata 模式（KRaft），以降低維運複雜度、移除擴展瓶頸、並減少潛在故障點。這也反映一個趨勢：系統傾向內建共識協定，而不是依賴外部協調服務。

可以考慮的替代方案

如果 ZooKeeper 不再像從前那麼主流，還有哪些選擇？

etcd 在雲原生環境很流行，也是 Kubernetes 的底層。它提供強一致性的分散式 key-value 儲存，有現代的 HTTP/JSON 與 gRPC API，並針對小資料集、高讀取量優化，適合設定管理與服務發現。

Consul（HashiCorp）不只做基本協調，還包含服務發現、健康檢查、key-value 儲存。它的特色是網路基礎設施自動化：能根據服務變化動態設定 load balancer 與防火牆，比 ZooKeeper 專注的協調範圍更廣。

雲端廠商方案如 AWS Parameter Store、Azure App Configuration、Google Cloud Datastore 提供託管協調服務，並與其他雲端服務整合。這些方案幾乎不用自己設定與維護，在系統設計面試裡是值得提到的實務替代方案。

隨著雲原生方案興起，很多雲端廠商都提供內建協調服務，讓你不必直接管理 ZooKeeper 這類共識系統。例如在 AWS 生態裡，AWS ECS 透過集中控制平面做容器編排，AWS CloudMap 簡化服務發現，Amazon MSK 則以全託管方式提供 ZooKeeper 功能。這些整合方案讓開發者能把精力放在應用上，而不是維護複雜的協調基礎設施。

限制

尤其在深入做基礎設施設計面試時，了解 ZooKeeper 的限制很重要。主要有幾點：

Hot Spotting：當很多客戶端 watch 同一個 ZNode（常見於 leader 選舉或鎖），伺服器可能被大量通知壓垮。在規模大時，熱門節點會變成瓶頸——想像聊天應用裡幾百萬使用者同時上線。

效能限制：ZooKeeper 的一致性模型讓寫入很貴，因為必須經由 leader 傳播到 quorum。它的記憶體儲存模型也限制資料量——ZNode 要維持在 1MB 以下，且整個資料集要能放進記憶體。

bulldoator

維運複雜度：ZooKeeper 需要仔細設定 Java 參數、磁碟配置，並持續監控 timeout 與連線數。正如某位維護者所說：「ZooKeeper 用起來簡單，但維運起來複雜。」

若你的設計需要存大量資料、承受極高寫入負載、或希望盡量降低維運複雜度，可能就要考慮其他方案。

那什麼時候該用 ZooKeeper？

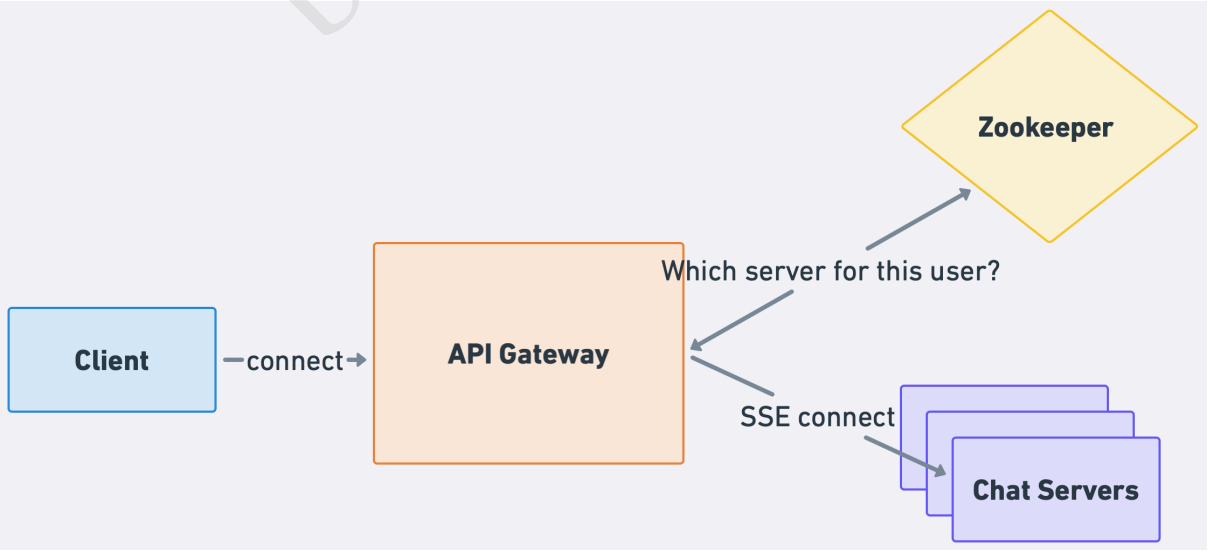
老實說，它通常不應該是你的第一選擇，但在面試裡還是有幾種情境適合提到 ZooKeeper。

Smart Routing

在聊天應用例子裡，我們討論了 ZooKeeper 如何幫訊息轉到正確的伺服器。但還有一個更進階的用法。就算我們用 pub/sub 廣播系統來送訊息、取代 ZooKeeper 做傳遞，仍有一個重要優化問題：限制每台伺服器訂閱的 channel 或 topic 數量。為了效能，我們希望同一個聊天室的使用者盡量在同一台伺服器上。也就是說，同一個聊天室的所有 websocket 連線盡可能由同一台伺服器處理。

這種「同地放置」策略在設計 Facebook Live Comments 或 YouTube Live Chat 這類系統時更重要。當幾百萬人同時看同一支直播，讓同一支直播的觀眾都連到同一台（或同一組）伺服器，可以最小化跨伺服器通訊。ZooKeeper 可以作為 API gateway 的協調點，維護「哪個聊天室或直播由哪台伺服器處理」的對應。當新使用者連線時，gateway 查 ZooKeeper，根據使用者的聊天室或直播 ID 決定最適合的伺服器。

實作上可能長這樣：每台伺服器在 ZooKeeper 註冊自己的容量與正在處理的房間列表。API gateway 收到新連線時，查 ZooKeeper 依使用者要進的房間找合適的伺服器，再把使用者導過去。若處理某熱門房間的所有伺服器都滿了，ZooKeeper 可以協助以協調的方式選出新伺服器來擴展該房間的容量。



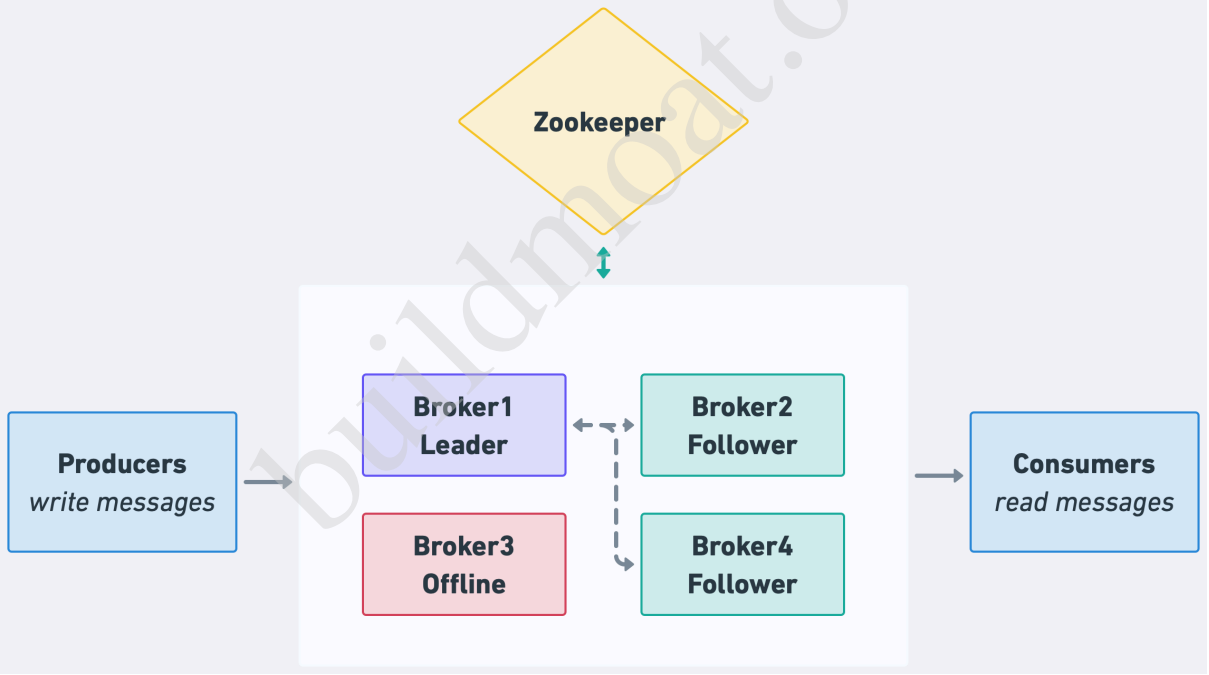
buildneat.org

這是進階主題，通常只會在 Staff+ 或深度討論分散式系統的資深工程師面試裡出現。

特定基礎設施設計題

在深入的基礎設施系統設計面試裡，例如「設計一個分散式訊息佇列」或「設計一個分散式任務排程器」，ZooKeeper 會特別相關。在這些題目裡，ZooKeeper 扮演系統的共識「大腦」，就像 Kafka 歷史上用它一樣。在分散式訊息佇列設計裡，ZooKeeper 會負責：

Broker 加入叢集時在 ZooKeeper 註冊，讓其他元件能發現它。建立 topic 時把 metadata 寫入 ZooKeeper，包括 partition 數量與複製因子。每個 partition 的 leader 選舉依賴 ZooKeeper 的 sequential ephemeral nodes，確保每個 partition 只有一個 broker 當 leader。Consumer group 在 ZooKeeper 追蹤成員與 partition 分配，以便在 consumer 加入或離開時做 rebalance。或許最重要的是，broker 故障偵測透過 ZooKeeper 的 ephemeral nodes——broker 掛掉時 session 過期、節點消失，觸發 partition 重新分配。



buildmoat.org

如前所述，這套模式在 KRaft 之前支撐了 Kafka 多年，類似做法也用在 HBase、Hadoop、Solr 等分散式系統。在設計這類系統時，ZooKeeper 提供久經考驗的解法，應對它們面對的複雜協調問題。

持久分散式鎖

Redis 可以處理很多情境的分散式鎖（例如票務系統），但當你需要階層式鎖、或有複雜依賴時，ZooKeeper 更合適。例如在分散式檔案系統裡，若需要巢狀取得鎖（例如鎖定目錄與其檔案）並避免死鎖，ZooKeeper 很適合。ZooKeeper 能同時對多個節點

維持 watch 通知，讓客戶端監控整個鎖的階層，並在鎖結構有任何變動時立即收到通知。這在資源有父子關係、且取得鎖必須尊重這些階層以避免死鎖、並在分散式元件間維持資料完整性的系統裡特別有價值。

總結

ZooKeeper 是一個分散式協調服務，協助分散式應用管理設定、命名與同步。它提供類似檔案系統的簡單介面，但具備管理設定、服務發現、leader 選舉、分散式鎖等進階能力。雖然現在有 etcd、Consul、雲端廠商方案等替代品，ZooKeeper 所代表的模式在分散式系統設計裡仍然非常根本。

但要小心。除非你是在做深入的基礎設施系統設計、需要討論如何讓多台伺服器謹慎協調，或是需要現代 load balancer 與內建服務發現工具無法提供的進階功能，否則在下次系統設計面試裡不要一上來就搬出 ZooKeeper。

buildmoat.org